esprit Se former autrement HONORIS UNITED UNIVERSITIES	Examen **DevOps** **Session Rattrapage**

Unité d'enseignement : Architecture N-Tiers

Module : DevOps

Documents autorisés : NON — Classes : 5 -ème

Connexion internet autorisée : NON — Nombre de pages : 6

Date : 13/12/2025 — Heure : 9h00 — Durée : 1h30

Toutes les réponses doivent être inscrites sur la feuille de réponse.

Partie 1 : QCM (5 pts)

1. Quelle est l'une des valeurs fondamentales du mouvement DevOps ?
A. La séparation stricte entre développeurs et opérationnels
B. L'automatisation de la documentation
C. La collaboration et la responsabilisation partagée
D. L'externalisation de tous les services

2 Une PersistentVolumeClaim (PVC) dans Kubernetes est utilisée pour **:**
A. Partager des variables d'environnement
B. Augmenter la RAM d'un pod
C. Persister les données d'un pod
D. Héberger le code source de l'image

3. Quel composant orchestre le placement des pods sur les nœuds ?
A. apiserver
B. scheduler
C. kubelet
D. kube-proxy

4. Quelle est la bonne séquence d'un pipeline CI/CD classique ?
A. Build → Deploy → Test → Analyse
B. Analyse → Test → Build → Deploy
C. Build → Test → Analyse code → Deploy
D. Test → Deploy → Analyse code→ Build

5. Le controller-manager dans Kubernetes est :

A. Un composant qui gère la communication réseau entre les pods
B. Un composant qui regroupe plusieurs contrôleurs assurant différentes tâches du cluster C. Un conteneur qui exécute directement les applications dans les pods D. Un outil externe permettant de monitorer les performances du cluster

6. Un artefact dans une chaîne DevOps désigne :
A. Une alerte de sécurité
B. Un fichier généré après une phase de build
C. Une ligne de commande
D. Une documentation utilisateur

7. Quel composant du Worker Node communique directement avec l'API Server du Control Plane ?
A. Kubelet
B. Kube Proxy
C. Container Runtime
D. Scheduler

8. Quelle commande Docker permet d'exécuter un conteneur à partir d'une image ? A. docker commit
B. docker build
C. docker run
D. docker built

9. Dans le pipeline CI/CD, la phase de "déploiement continu" consiste à : A. Compiler le code
B. Livrer automatiquement les modifications en production après validation C. Effectuer des tests manuels
D. Écrire des spécifications

10. Quel composant est responsable de maintenir l'état désiré du cluster et de gérer les pods sur les nœuds ?
A. kubelet
B. apiserver
C. controller-manager
D. etcd

Partie 2 : (7 pts)

1. Un conteneur est en cours d'exécution à partir de l'image springboot-app:latest. Quelle commande permet d'entrer dans le conteneur avec un shell ?(0.25pt)

docker _____

2. Un pod webapp dans le namespace production est en état CrashLoopBackOff. Quelle commande permet de voir les logs pour identifier le problème ? (0.25pt)
kubectl _____-n production

3. citer 4 types de problèmes remontés par SonarQube ? (1pt)

4. Quelle est la différence entre virtualisation lourde et virtualisation légère ? (0.25pt) 5.

Dans l'architecture Kubernetes, citez les principaux composants du Control Plane : (1pt)

6. Quel est le rôle principal de Prometheus dans un cluster Kubernetes ? (0.25pt)

Pour chaque description ci-dessous, indiquez le nom du concept ou de l'outil correspondant dans l'écosystème Docker/Kubernetes/CI-CD.

1. C'est un point d'exécution dans Kubernetes qui regroupe un ou plusieurs conteneurs. (0.5)

2. C'est un fichier texte décrivant comment construire une image Docker. (0.5)

3. C'est un ensemble d'étapes automatisées permettant de compiler, tester et déployer l'application (artefact) issue du code dans un processus DevOps. (0.5)

4. C'est un objet Kubernetes qui expose un ensemble de pods via un point d'accès réseau stable. (0.5)

5. C'est un outil d'intégration continue utilisé pour automatiser les étapes de construction, le test et le déploiement. (0.5)

6. Quel objet Kubernetes représente un stockage persistant utilisé par un conteneur pour conserver ses données ? (0.5)

7. C'est un composant s'exécutant sur un nœud qui gère le routage du trafic réseau vers les pods et assure la communication réseau à l'intérieur du cluster Kubernetes. (0.5)

8. Quel est le nom du registre distant où l'on stocke et partage des images Docker (0.5)

Étude de cas – DevOps (8 pts)

Vous travaillez en tant qu'ingénieur DevOps pour une équipe de développement qui a produit une application Spring Boot nommée springboot-app.
Le but est de mettre en place un pipeline CI/CD complet sous Jenkins afin d'automatiser les étapes suivantes :

1. Récupération du code source depuis le dépôt Git.
2. Construction du projet Maven
3. Analyse de la qualité du code avec SonarQube.
4. Construction et publication d'une image Docker dans le registre docker.io/moncompte.
5. Déploiement automatique de l'application sur un cluster Kubernetes dans le namespace production.

Éléments techniques suivants sont mis à disposition :

Langage : **Java 17 – Spring Boot**

Image Docker de base : **openjdk:17-jdk**

Artifact généré : **target/app.jar**

Port de l'application : **8080**

Registre Docker : **docker.io/moncompte**

Namespace Kubernetes : **production**

Base de données : MySQL service mysql-service:3306

· user : root

· password : password

Git Repo : **https://github.com/moncompte/springboot-app.git**

Branche : **main**

Fichiers Kubernetes à appliquer :

· k8s/deployment.yaml

· k8s/service.yaml

Vérification : afficher pods + services du namespace

1- Completer le fichier Dockerfile

2- Compléter le fichier Jenkinsfile complet avec les étapes CI/CD

(Build → Analyse → Docker → Déploiement → Vérification).

Dockerfile à compléter (1.5):

```
FROM TODO_1

WORKDIR /app

COPY TODO_2/app.jar /app/app.jar

EXPOSE TODO_3

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Jenkinsfile à compléter (6.5) :

```
pipeline  {
 agent any
 tools {
 jdk 'JDK_17'
 maven 'Maven_3'
 }
 environment {
 REGISTRY = " TODO_4 "
 APP_NAME = "springboot-app"
 KUBE_NAMESPACE = " TODO_5 "
 }
 stages {
 stage('Checkout') {
 steps {
 checkout([branches: [[name: '*/main']], userRemoteConfigs:
[url:'https://github.com/moncompte/springboot-app.git']]]) }
 }
 stage('Build') {
 steps {
 sh 'mvn “TODO_6 ”
 }
 stage('Analyse Qualité') {
 steps {
 withSonarQubeEnv(Sonarqube_serveur) {
 sh ' TODO_7 '
 }}
 stage('Build Docker Image') {
 steps {
```

```
sh '''
docker TODO_8 $REGISTRY/$APP_NAME:latest .
}
stage(Push Docker Image') {
steps {
sh '''
docker TODO_9 $REGISTRY/$APP_NAME:latest
}}
stage('Déploiement Kubernetes') {
steps {
sh '''
kubectl apply -f k8s/TODO_10 -n $KUBE_NAMESPACE

kubectl apply -f k8s/TODO_11 -n $KUBE_NAMESPACE
}}
stage('Vérification Déploiement') {
steps {
sh '''
kubectl TODO_12 -n $KUBE_NAMESPACE // état réel des pods
 kubectl TODO_13 -n $KUBE_NAMESPACE // vérifier que les services sont exposés }}
}}
```

Bon travail !